

URL Shortener SDLC

Agile Iterative Development Approach

- **Shortening was built first**
- **Persistence was then added**
- **Expiration added**
- **Rate Limiting added**
- **Click tracking added**
- **Front-end added**

Overview

An application capable of taking long URLs, and then shortening them. When the short URL is clicked, click count is tracked and then the user is redirected to the original URL.

SDLC Steps:

1. Requirements Gathering

Functional Requirements:

- Generate Short Urls
- Prevent app from being overwhelmed through rate limiting
- Redirect to original Url when short Url clicked
- Persistence through storing Urls in a database
- User can opt for an expiration date on the short Url
- Error handling
- Click tracking
- Front-end

Non-Functional Requirements:

- Handle a high number of possible combinations
- Prevent Abuse through the rate limiting
- Cross-Origin Resource Sharing (CORS) Support
- Secure Cookies through the use of SameSite=None, Secure

2. System Design

High-Level Design Decisions

- Angular for Front-End
- ASP .NET Web API for Back-End
- SQLite for Database
- EF Core ORM
- Cookie-based Anonymous Identity
- Custom in-memory rate limiter
- RESTful Endpoints

Design Decisions Included:

- 6-Character Alphanumeric Codes
- Structured JSON error responses

3. Implementation

3

Frontend:

- Reactive forms
- Error handling via `HttpErrorResponse`
- Modal UI
- Conditional rendering
- Clipboard API

Backend:

- Async EF Core operations
- Custom rate limiting algorithms (Token Bucket)
- Cookie-based user tracking
- Expiration validation
- Click count tracking
- CORS configuration

4. Testing

Manual testing via:

- Swagger
- Angular frontend
- Browser DevTools (cookies + network)
- Testing 429 responses
- Testing expired links
- Testing invalid URLs
- Testing cross-origin credentials
- Database persistence
- Duplicate URL handling
- Rate limiting boundaries

5. Deployment

Designed with production deployment considerations in mind.